# MISAKA Network

*A Post-Quantum Native Layer 1 Blockchain with Uncertified DAG Consensus*

Itsu

*MISAKA Network — Core Development*
misakabtc@gmail.com
https://misakachain.com

Whitepaper Version 1.0

April 2026

**Abstract**

This document specifies **MISAKA Network**, a Layer 1 blockchain whose entire signing and key-exchange surface is built exclusively from NIST-standardized lattice cryptography. Every user-visible signature is an ML-DSA-65 (FIPS 204) signature; every validator-to-validator session is keyed by an ML-KEM-768 (FIPS 203) handshake; no elliptic-curve primitive appears anywhere in the consensus critical path. The ledger is a transparent UTXO model with a SHA3-256 MuHash incremental commitment over the unspent set. Consensus is provided by a Narwhal/Bullshark uncertified DAG engine, ported from the Mysticeti reference implementation, that commits blocks in three message delays under good network conditions. The active committee consists of 21 super representatives selected by an integer square-root weighting of staked supply, with safety enforced by a stake-weighted Byzantine quorum threshold of strictly more than two thirds of total active stake. This v1.0 release combines the previously separate Phase 1, Phase 2, and Phase 3 working drafts into a single document with unified cross-references, an auto-generated table of contents, and continuous page numbering.

# Contents


**1. Introduction** **6**

1.1. Contributions . . . 6

1.2. Document structure . . . 6

**2. System Model and Assumptions** **6**

2.1. Validators, epochs, and stake . . . 6

2.2. Adversary model . . . 7

2.3. Quantum adversary . . . 7

2.4. Network model . . . 7

2.5. Stake-weighted quorum . . . 7

2.6. Goals . . . 7

**3. Cryptographic Primitives** **8**

3.1. ML-DSA-65 (FIPS 204) . . . 8

3.2. ML-KEM-768 (FIPS 203) . . . 8

3.3. Hash functions . . . 8

3.4. Notation . . . 9

**4. Data Model** **9**

4.1. Primitive types . . . 9

4.2. Addresses . . . 9

4.3. Outputs and outpoints . . . 9

4.4. Lock scripts and unlock witnesses . . . 9

4.5. Inputs and transactions . . . 9

4.6. Transaction identifier and sighash . . . 10

4.7. UTXO set commitment (MuHash) . . . 10

**5. Authenticators** **11**

**6. Transaction Effects** **11**

6.1. Effect structure . . . 11

6.2. Determinism . . . 11

6.3. Effect digest . . . 11

6.4. Failed transactions . . . 12

**7. System Operation Overview** **12**

7.1. Components . . . 12

7.2. The transaction lifecycle . . . 12

**8. Validator Persistent State** **13**

8.1. The UTXO store . . . 13

8.2. The DAG store . . . 13

8.3. The committed-leader log . . . 13

8.4. The validator-set store . . . 13

8.5. The supply tracker . . . 13

8.6. The write-ahead log . . . 13

**9. Authority Base Operation** **14**

9.1. Admitting a transaction . . . 14

9.2. Receiving a block . . . 14

9.3. Proposing a block . . . 14

9.4. Committing a leader and executing . . . 15

**10. Narwhal/Bullshark Consensus** **15**

10.1. Block structure . . . 15

10.2. Block validity rules . . . 16

10.3. Commit rule . . . 16

10.4. Linearization . . . 17

10.5. Liveness timeout . . . 17

**11. Epoch Change and Reconfiguration** **17**

11.1. Epoch length . . . 17

11.2. Validator selection . . . 17

11.3. Voting weight versus reward weight . . . 17

11.4. Epoch transition procedure . . . 18

11.5. Cross-epoch safety . . . 18

**12. Light Clients** **18**

12.1. Header chain . . . 18

12.2. Trust anchor: the validator set . . . 18

12.3. Quorum certificate verification . . . 19

12.4. Absence of signature aggregation . . . 19

**13. Tokenomics** **19**

13.1. Maximum supply and units . . . 19

13.2. Emission curve . . . 19

13.3. Coinbase distribution . . . 20

13.4. Fee model . . . 20

13.5. Staking and bonding . . . 20

**14. Scaling and Latency** **20**

14.1. Block-level overhead . . . 20

14.2. Network-level overhead . . . 20

14.3. Verification CPU . . . 21

14.4. Latency . . . 21

14.5. Throughput . . . 21

**15. Security Analysis** **21**

15.1. Safety . . . . . 21

15.2. Post-quantum properties . . . . . 22

15.3. Snapshot durability of supply . . . . . 22

**16. Related Work** **22**

**17. Conclusion** **23**

**References** **23**

**Appendix A. Script Opcode Reference** **23**

**Appendix B. Constants** **24**

**Appendix C. Acknowledgments** **24**

# 1. Introduction

Public blockchains in production today derive their security from the hardness of the elliptic-curve discrete logarithm problem. Bitcoin uses secp256k1; Ethereum and the EVM ecosystem use secp256k1 and BLS12-381; Solana, Sui, and the Move ecosystem use Ed25519 and BLS12-381; Cosmos uses Ed25519. Shor's algorithm [1] reduces this problem to polynomial time on a sufficiently capable quantum computer, which means that every signature currently securing these chains can in principle be forged once such a computer exists. Public keys that have been revealed on-chain — which is most of them, since spending a UTXO or sending a transaction publishes the key — are harvestable today and breakable later.

The standard industry response is to plan a future migration to post-quantum cryptography (PQC) via a hard fork. This response is structurally insufficient. A hard fork can stop new vulnerable signatures from being produced, but it cannot retroactively protect outputs whose public keys have already been published, nor can it protect a network transcript that an adversary has already recorded. Any chain that wants to provide settlement guarantees that survive the arrival of a cryptographically relevant quantum computer must be post-quantum from genesis.

MISAKA Network is a Layer 1 blockchain designed under exactly this constraint. It is written in Rust, runs a transparent UTXO ledger with a fixed maximum supply of 10,000,000,000 MISAKA, and is secured by a Narwhal/Bullshark DAG consensus engine ported from Sui's Mysticeti reference implementation [2]. Every signature in the system — user transactions, validator block proposals, DAG votes, checkpoint attestations, validator-set update certificates — is an ML-DSA-65 signature [3]. Every TCP session between validators is keyed by an ML-KEM-768 handshake [4] and encrypted with ChaCha20-Poly1305. There is no secp256k1 code path, no Ed25519 code path, no BLS code path, and no pairing-based primitive anywhere in the consensus critical section. This property is checked by a build-time linter that rejects any introduction of such a dependency.

## 1.1. Contributions

This specification makes four contributions.

**(C1)** We give the first complete specification of a Layer 1 blockchain whose consensus engine, peer-to-peer transport, and user-level signing scheme are all exclusively lattice-based, with explicit absence of elliptic-curve cryptography from the consensus critical path.

**(C2)** We adapt the Mysticeti uncertified DAG commit rule [2] to a UTXO ledger authenticated by ML-DSA-65, and analyze the resulting overhead, which we show is dominated by the fixed 3309-byte ML-DSA-65 signature size rather than by any increase in protocol complexity.

**(C3)** We define a snapshot-durable supply commitment structure (the *SupplyTracker*) that records emitted, circulating, staked, and burned supply as first-class persistent state, and prove (Theorem 7) that this design eliminates a class of replay-based supply-cap bypass attacks possible against tokenomics implementations that recompute supply on validator restart.

**(C4)** We define an integer-square-root stake-weighting rule for a fixed 21-slot active committee, computed using a deterministic branchless integer square root over u128. The rule reduces the marginal voting power of high-stake validators relative to a linear weighting while preserving strict monotonicity in stake and Sybil resistance.

## 1.2. Document structure

This document is organized in three parts. **Part I** (Sections 2–6) defines the protocol model: the system model and threat assumptions, the cryptographic primitives, the data types of the ledger, the address and authenticator format, and the effects produced by transaction execution. **Part II** (Sections 7–11) specifies the validator state machine, authority operation as numbered procedures, the Narwhal/Bullshark consensus engine, and the DPoS reconfiguration protocol. **Part III** (Sections 12–17) covers the light-client protocol, tokenomics, scaling and latency analysis, the security analysis with formal theorems, related work, and conclusions. Three appendices follow: a script opcode reference, a constants table, and acknowledgments.

# 2. System Model and Assumptions

## 2.1. Validators, epochs, and stake

MISAKA operates in a sequence of *epochs*. We index epochs by $e \in \mathbb{N}$ starting from $e = 0$ at genesis. For each epoch $e$ we define a tuple

$$C_e = (V_e, S_e)$$

where $V_e$ is the ordered set of validator identities active in epoch $e$, and $S_e : V_e \to \mathbb{N}$ is the function assigning each validator its active stake in base units of MISAKA. We refer to $C_e$ as the *committee* for epoch $e$. The committee is fixed for the duration of the epoch and is updated only at

epoch boundaries by the reconfiguration protocol of Section 11.

The MISAKA active committee size is fixed at $|V_e| = 21$ for every epoch. The minimum active stake is MIN_STAKE_ACTIVE = $10^7$ MISAKA (equivalently $10^{16}$ base units, since 1 MISAKA = $10^9$ base units). A backup set of additional validators with at least $10^6$ MISAKA staked is maintained outside the active committee for rotation purposes; these validators do not participate in consensus and are specified in Section 11.

## 2.2. Adversary model

Let $T_e = \Sigma_{v \in V} S_e(v)$ denote the total active stake in epoch $e$. A computationally bounded adversary may, at the start of epoch $e$, statically corrupt an arbitrary subset $B_e \subseteq V_e$ of validators subject to the constraint

$$\Sigma_{v \in B_e} S_e(v) < T_e / 3.$$

We refer to validators in $B_e$ as *Byzantine*; they may deviate from the protocol arbitrarily, including signing equivocating messages, withholding messages, or colluding with one another. The remaining validators in $V_e \setminus B_e$ are *honest* and follow the protocol exactly.

Note that this is a *stake-weighted* Byzantine bound, not a count-weighted bound. The condition $|B_e| \leq 6$ (which would be the count-weighted analogue with $n = 21$) is neither necessary nor sufficient: the adversary may corrupt fewer validators if they collectively hold more than one third of the stake, and may corrupt more validators if they collectively hold less. This matches the model used by Sui [9] and Cosmos and is the appropriate model for any chain in which validator weight is not uniform.

## 2.3. Quantum adversary

We strengthen the adversary in one direction relative to standard BFT models: the adversary is assumed to have access to a *cryptographically relevant quantum computer* (CRQC). Concretely, the adversary may solve the elliptic-curve discrete logarithm problem, the integer factorization problem, and any classical discrete-log-based problem in expected polynomial time. The adversary is *not* assumed to efficiently solve the Module Learning-With-Errors (MLWE) or Module Short-Integer-Solution (MSIS) problems on the parameters specified by FIPS 203 and FIPS 204; these are the only computational hardness assumptions on which MISAKA's safety relies.

## 2.4. Network model

Honest validators are connected by authenticated point-to-point links established by the ML-KEM-768 handshake of Section 3. The network is partially synchronous in the sense of Dwork, Lynch, and Stockmeyer: there exists an unknown Global Stabilization Time (GST) and a finite delay bound $\Delta$ such that any message sent by an honest validator at time $t$ is delivered to its destination by time $\max(\text{GST}, t) + \Delta$. Before GST the network is fully asynchronous and the adversary may delay messages arbitrarily. We require safety to hold under asynchrony and liveness to hold after GST.

## 2.5. Stake-weighted quorum

Throughout the specification we make extensive use of the notion of a *stake quorum*. A set of validators $Q \subseteq V_e$ is a stake quorum in epoch $e$ if and only if

$$\Sigma_{v \in Q} S_e(v) > (2 / 3) \cdot T_e.$$

The implementation rounds this up to $\lfloor (2 \cdot T_e / 3) \rfloor + 1$ in integer arithmetic to avoid floating-point. By a standard quorum-intersection argument, any two stake quorums intersect in a set whose total stake exceeds $T_e / 3$, and therefore contains at least one honest validator under the bound of Section 2.2. This intersection lemma is the single most-used proof tool in the security analysis of Section 15.

**Remark on count-based quorums.** The Mysticeti specification [2] is written assuming a uniform-stake committee of size $n = 3f + 1$, with a quorum of $2f + 1$ validators. MISAKA uses the stake-weighted generalization of this rule throughout. Where we cite a Mysticeti lemma in this document we are citing the stake-weighted variant of that lemma; the proofs go through without modification because they only use quorum intersection, which the stake-weighted formulation preserves.

## 2.6. Goals

MISAKA is a Byzantine Atomic Broadcast (BAB) protocol [5], [6]. We require the four standard properties:

**Agreement.** If an honest validator $v_i$ delivers a transaction $tx$, then every honest validator $v_j$ eventually delivers $tx$.

**Integrity.** Each honest validator delivers each transaction at most once, and only if some validator previously broadcast it.

**Validity.** If an honest validator broadcasts a transaction, then every honest validator eventually delivers it.

**Total order.** If two honest validators $v_i$ and $v_j$ deliver transactions $tx$ and $tx'$, then they deliver them in the same relative order.

We additionally require **post-quantum forward secrecy**: a network transcript recorded today must remain confidential even if the adversary subsequently acquires a CRQC.

## 3. Cryptographic Primitives

MISAKA uses three cryptographic primitives in its consensus and ledger code: ML-DSA-65 for digital signatures, ML-KEM-768 for key encapsulation, and SHA3-256 for collision-resistant hashing. BLAKE3 is additionally used for non-consensus paths (mempool deduplication, write-ahead-log checksums) where throughput matters more than standardization. This section defines each primitive and the security assumption it imposes.

### 3.1. ML-DSA-65 (FIPS 204)

ML-DSA is the Module-Lattice-based Digital Signature Algorithm standardized by NIST in FIPS 204 [3], based on the CRYSTALS-Dilithium submission. MISAKA uses the parameter set ML-DSA-65, corresponding to NIST security category 3 (approximately 192-bit classical and 128-bit post-quantum security). The scheme is a triple of algorithms

```
KeyGen() → (pk, sk)
Sign(sk, m) → σ
Verify(pk, m, σ) → {0, 1}
```

with concrete sizes |pk| = 1952 bytes, |sk| = 4032 bytes, and |σ| = 3309 bytes. We assume the scheme is existentially unforgeable under adaptive chosen-message attack (EUF-CMA) against any polynomial-time quantum adversary, under the MLWE and MSIS assumptions on the FIPS 204 parameter set. All proofs of authentication-related properties in Section 15 reduce to this assumption.

ML-DSA-65 is used unmodified for: (i) the unlocking signature on every UTXO input (Section 4); (ii) the author signature on every consensus block (Section 10); (iii) the operator-key authentication on every validator-to-validator handshake; and (iv) checkpoint attestations at epoch boundaries.

### 3.2. ML-KEM-768 (FIPS 203)

ML-KEM is the Module-Lattice-based Key Encapsulation Mechanism standardized by NIST in FIPS 203 [4], based on the CRYSTALS-Kyber submission. MISAKA uses the parameter set ML-KEM-768, corresponding to NIST security category 3. The scheme is a triple

```
KeyGen() → (pk, sk)
Encaps(pk) → (ct, K)
Decaps(sk, ct) → K
```

where K is a 32-byte shared secret. We assume the scheme is IND-CCA2 secure against polynomial-time quantum adversaries under the MLWE assumption.

MISAKA establishes every TCP session between validators by running an ML-KEM-768 encapsulation, deriving an AEAD key pair via HKDF-SHA3-256 from the encapsulated secret, and authenticating both endpoints with ML-DSA-65 signatures over the handshake transcript. The resulting session is then encrypted with ChaCha20-Poly1305. We deliberately avoid hybrid X25519+ML-KEM constructions: any elliptic-curve component in the handshake exposes the session to harvest-now-decrypt-later attacks once a CRQC exists, which would defeat the entire purpose of the post-quantum design. The session lifetime is bounded by a configurable parameter MAX_SESSION_LIFETIME_SECS; the testnet default is 86400 seconds and the recommended validator value is 14400 seconds, after which the handshake is re-run.

### 3.3. Hash functions

MISAKA uses two hash functions, each in a clearly delimited scope.

**SHA3-256** [7] is the consensus-visible hash function. It is used for: transaction identifiers, block hashes, Merkle-tree branches, the MuHash UTXO accumulator (Section 4.7), address checksums, the HKDF used in session-key derivation, and any value that is signed by an ML-DSA-65 signature. We assume SHA3-256 is collision-resistant, preimage-resistant, and second-preimage-resistant against polynomial-time quantum adversaries with the standard Grover speedup applied.

**BLAKE3** is used only on non-consensus paths: mempool deduplication, write-ahead-log block checksums, and internal caching keys. No value hashed under BLAKE3 is ever placed into the consensus state, signed, or referenced from a block. The motivation is throughput: BLAKE3 is approximately an order of magnitude faster than SHA3-256 on commodity hardware, and the non-consensus paths benefit from this without adding a hard dependency on BLAKE3 for safety. The build-time linter that enforces post-quantum purity also enforces this scope boundary.

### 3.4. Notation

We write $H(\cdot)$ for SHA3-256. We write $\mathrm{Sign}_{sk}(m)$ and $\mathrm{Verify}_{pk}(m, \sigma)$ for ML-DSA-65 operations, omitting the algorithm name where clear from context. Concatenation of byte strings is denoted by ‖. The notation ⟦·⟧ denotes a deterministic byte-level encoding (specified in Section 4.6) used as the input to $H$.

## 4. Data Model

MISAKA is a transparent UTXO ledger. The ledger state at any point is fully described by the set of unspent transaction outputs together with a small amount of auxiliary state (supply tracker, validator set, epoch number). This section defines the on-chain types in a form that is intended to be directly translatable to a Rust struct definition or to a wire-format schema.

### 4.1. Primitive types

Table 1 lists the primitive types used by the rest of the data model.

| Type | Definition |
|---|---|
| U64 | 64-bit unsigned integer, little-endian on the wire. |
| U128 | 128-bit unsigned integer, used for stake and supply. |
| Bytes32 | Fixed-length 32-byte string. |
| ChainID | U32, 1 = mainnet, 2 = testnet, others reserved. |
| Round | U64, monotonic logical round number, starts at 0. |
| Epoch | U64, monotonic epoch number, starts at 0. |
| Hash | Bytes32, output of H(·) = SHA3-256. |
| MLDSAPubKey | 1952-byte ML-DSA-65 public key. |
| MLDSASig | 3309-byte ML-DSA-65 signature. |

*Table 1: Primitive types.*

### 4.2. Addresses

An *address* in MISAKA is the SHA3-256 hash of an ML-DSA-65 public key, wrapped in a chain-bound textual envelope. Formally:

| Field | Type | Description |
|---|---|---|
| raw | Bytes32 | H("MISAKA:addr:v2:" ‖ pk), where pk is the ML-DSA-65 public key. |
| chain_id | ChainID | The chain identifier this address is bound to. |
| checksum | U16 | First 2 bytes of H("MISAKA:addr:checksum:v2:" ‖ chain_id_le4 ‖ hex(raw)). |

*Table 2: Address structure.*

The textual encoding is misaka1 ‖ hex(raw) ‖ hex(checksum). Because chain_id is included in the checksum preimage, the same raw bytes produce a syntactically distinct textual address on testnet (chain_id = 2) and on mainnet (chain_id = 1). This eliminates the entire class of cross-chain address-confusion attacks at the validation layer.

### 4.3. Outputs and outpoints

| Field | Type | Description |
|---|---|---|
| value | U64 | Amount in base units (1 MISAKA = $10^9$ base units). |
| lock | LockScript | Spending condition; see Section 4.4. |

*Table 3: TxOut structure.*

An *outpoint* identifies a specific output produced by a specific transaction. It is the pair (txid, index) where txid is the hash of the transaction that produced the output and index is the zero-based position of the output in that transaction's output list.

### 4.4. Lock scripts and unlock witnesses

Every output carries a *lock script*, a short program in the MISAKA script language that defines the conditions under which the output may be spent. The script language is a stack machine in the style of Bitcoin Script and Kaspa Script, restricted to a small set of opcodes and extended with post-quantum signature checks. The two principal opcodes are:

**OP_CHECKSIG_MLDSA**: pops a 1952-byte public key and a 3309-byte signature from the stack, computes the transaction sighash $z$ as defined in Section 4.6, and pushes 1 if $\mathrm{Verify}_{pk}(z, \sigma) = 1$ and 0 otherwise.

**OP_CHECKSIGVERIFY_MLDSA**: identical to OP_CHECKSIG_MLDSA but immediately fails the script if verification returns 0.

No legacy OP_CHECKSIG opcode invoking secp256k1 or Ed25519 exists in the MISAKA script set, and any attempt to deserialize a script containing such an opcode is rejected at the consensus layer with a deterministic error code. The complete v1 opcode set is given in Appendix A.

### 4.5. Inputs and transactions

| Field | Type | Description |
|---|---|---|
| outpoint | (Hash, U32) | The (txid, index) pair this input consumes. |
| unlock | UnlockWitness | Witness satisfying the lock script of the consumed output. |
| sequence | U32 | Per-input sequence number; reserved for future use, must be 0xFFFFFFFF in v1. |

*Table 4: TxIn structure.*

| Field | Type | Description |
|---|---|---|
| version | U16 | Transaction format version, currently 1. |
| chain_id | ChainID | The chain on which this transaction is valid. |
| inputs | Vec<TxIn> | Non-empty for non-coinbase, empty for coinbase. |
| outputs | Vec<TxOut> | At least one output. |
| lock_time | U64 | Earliest round number at which this transaction may be included; 0 for no lock. |
| fee | U64 | Explicit fee in base units; must equal sum(inputs) − sum(outputs). |

*Table 5: Transaction structure.*

A transaction is *well-formed* if its serialization is canonical (no trailing bytes, fixed integer encodings, lexicographically ordered fields), if every output value fits in U64, if the explicit fee field matches the input/output difference, and if chain_id matches the validator's local chain identifier. A well-formed transaction is *valid* at ledger state Σ if every input outpoint exists in Σ, every input unlock witness satisfies the corresponding lock script under the sighash defined below, and the sum of input values is at least the sum of output values plus the explicit fee.

## 4.6. Transaction identifier and sighash

The *transaction identifier* is

$$\texttt{txid} = \texttt{H("MISAKA:txid:v1:"} \parallel \llbracket \texttt{tx_without_witnesses} \rrbracket\texttt{)}$$

where ⟦tx_without_witnesses⟧ is the canonical serialization of the transaction with every unlock field replaced by a zero-length placeholder. This makes txid independent of the witness data, which is essential to prevent third-party transaction malleability.

The *sighash* for input *i* of transaction *tx* is

$$\texttt{sighash}_i = \texttt{H("MISAKA:sighash:v1:"} \parallel \texttt{chain_id} \parallel \texttt{txid} \parallel \texttt{i} \parallel \texttt{lock_script}_i\texttt{),}$$

where $\texttt{lock_script}_i$ is the lock script of the output consumed by input *i*. Including the lock script binds the signature to the specific output being spent and prevents witness reuse across outputs that share an address. Including the chain identifier prevents cross-chain transaction replay independently of the address format.

## 4.7. UTXO set commitment (MuHash)

The validator's view of the unspent transaction output set is summarized by an incremental multiset hash known as *MuHash*. The MISAKA implementation uses the SHA3-256 based variant defined in misaka-muhash. MuHash represents a multiset of byte strings as the product of their hashes in a large finite group; insertion and deletion of an element correspond to multiplication and division in that group, both of which are constant-time. The state commitment for the UTXO set after applying a block is therefore computable in time linear in the number of inputs and outputs touched by that block, rather than linear in the size of the entire UTXO set.

Concretely, every UTXO is canonically encoded as ⟦outpoint ‖ value ‖ lock_script⟧, hashed under SHA3-256, and accumulated into the validator's MuHash state. The 32-byte digest of this state is included in every block under the field name state_root. Two honest validators that have applied the same sequence of committed transactions starting from the same genesis state will produce bit-identical state_root values regardless of execution order, because MuHash is commutative. Any validator whose locally computed state_root diverges from the committed value halts and emits a fork alert; this is enforced unconditionally and is the mechanism by which independent execution verification is performed.

## 5. Authenticators

Section 4.2 defined the structure of an address. This section defines the *authenticator*: the data carried by an unlock witness that proves a transaction was authorized by the holder of the private key corresponding to an address.

An authenticator for the standard single-signature lock is a pair

```
Auth = (pk : MLDSAPubKey, σ : MLDSASig)
```

and is accepted by the script OP_CHECKSIG_MLDSA over input $i$ if and only if both of the following hold:

**(A1)** The address derived from pk via $H$("MISAKA:addr:v2:" ‖ pk) equals the address committed in the lock script being satisfied.

**(A2)** $\text{Verify}_{pk}(\text{sighash}_i, \sigma) = 1$, where $\text{sighash}_i$ is the value defined in Section 4.6.

Multi-signature locks and threshold locks are reserved for the v2 script extension and are not part of the v1 specification described here. The intended encoding is a MuSig-style aggregation over ML-DSA-65 keys, but no production-grade aggregation scheme for ML-DSA exists at the time of this draft and we therefore defer the specification until one does.

## 6. Transaction Effects

When a transaction is executed against a ledger state, it produces a structured value called its *effects*. The effects record exactly what the transaction did, in a form that can be hashed, signed, and reproduced by any independent executor. They are the unit over which state_root is computed and the unit that light clients verify (Section 12).

### 6.1. Effect structure

| Field | Type | Description |
|---|---|---|
| txid | Hash | Identifier of the transaction that produced these effects. |
| status | Status | One of {Success, Failure(code)}; only Success commits state changes. |
| consumed | Vec<OutPoint> | Outpoints removed from the UTXO set. |
| created | Vec<(OutPoint, TxOut)> | New outputs added to the UTXO set, with their assigned outpoints. |
| fee_paid | U64 | Amount in base units transferred to the fee pool of the committing block. |
| epoch | Epoch | Epoch in which the transaction was committed. |
| round | Round | Round in which the transaction's containing block was committed. |

*Table 6: Transaction effects.*

### 6.2. Determinism

Effect computation is required to be deterministic: two honest validators executing the same transaction against the same starting state must produce bit-identical effects. This rules out floating-point arithmetic, non-deterministic iteration over hash maps, system clock reads, and any other source of platform-dependent behavior. The MISAKA reference implementation enforces this through three mechanisms: (i) all monetary arithmetic uses checked u128 operations rather than floats; (ii) all iteration over collections is performed in a canonical order (lexicographic by key); and (iii) the integer square root used in stake weighting is implemented as a branchless binary search rather than as a libm call.

### 6.3. Effect digest

The *effect digest* is

```
digest = H("MISAKA:effect:v1:" ‖ effect).
```

Effect digests are aggregated by the block producer into the effects_root field of each block, computed as the SHA3-256 Merkle root of the digests of all transactions executed in that block. Light clients (Section 12) verify individual transaction effects by checking a Merkle inclusion proof against this root, without needing to download the full block payload.

### 6.4. Failed transactions

A transaction whose script verification fails, whose input values do not cover its output values plus fee, or whose lock_time has not yet been reached is committed with status Failure(code). A failed transaction consumes no UTXOs, creates no UTXOs, and pays no fee, but still occupies a slot in the block ordering and still produces an effect record. This is the same convention used by Sui [9] and is necessary to ensure that two validators agree on whether a transaction was attempted, even if they disagree on whether it would have succeeded under some alternative ordering.

## 7. System Operation Overview

Part I of this specification (Sections 2–6) defined the data model: what a transaction is, what an address is, what an effect is. Part II (Sections 7–11) defines what validators *do* with these objects. This section gives a high-level overview of the validator state machine and its principal interactions; Sections 8–11 specify each subsystem in detail.

### 7.1. Components

Each MISAKA validator runs five logical components, implemented as separate Rust crates in the reference code but connected through a single in-process address space:

**(1) The peer-to-peer transport** (misaka-p2p) maintains authenticated, encrypted TCP sessions to all other validators. Sessions are established by an ML-KEM-768 handshake (Section 3.2) and use ChaCha20-Poly1305 for record encryption.

**(2) The mempool** (misaka-mempool) buffers well-formed but not-yet-included transactions. It enforces fee-rate ordering, deduplication by transaction identifier, and a per-sender admission policy.

**(3) The consensus engine** (misaka-dag) is a port of the Mysticeti uncertified DAG protocol [2]. It produces signed blocks, gossips them over the P2P transport, interprets the resulting DAG to decide which proposer slots commit, and emits a totally ordered sequence of committed leader blocks to the execution layer. Section 10 specifies it in full.

**(4) The execution engine** (misaka-execution) consumes the committed sequence of leader blocks, applies the contained transactions to the UTXO set in deterministic order, and updates the MuHash state commitment. It is the only component that mutates ledger state.

**(5) The storage layer** (misaka-storage, misaka-database) persists the UTXO set, the supply tracker, the validator set, the consensus DAG, and the write-ahead log used for crash recovery. Storage is the only durable component; everything else can be reconstructed from it.

### 7.2. The transaction lifecycle

A transaction's path through MISAKA from client submission to final commit consists of seven steps:

**(L1)** The client constructs a transaction using the data model of Section 4, signs each input with its ML-DSA-65 secret key over the appropriate sighash, and submits the transaction to one or more validator RPC endpoints.

**(L2)** The receiving validator runs the well-formedness checks of Section 4.5, looks up each consumed outpoint in its local UTXO set, verifies each unlock witness against the corresponding lock script, and admits the transaction to the mempool if all checks pass.

**(L3)** When the validator's turn comes to propose a block, it drains a fee-rate-ordered batch of transactions from the mempool, includes them in the payload of a new block, signs the block with its ML-DSA-65 operator key, and broadcasts it to all other validators via the P2P transport.

**(L4)** Other validators receive the block, verify the author signature, verify that the block's parent references satisfy the validity rules of Section 10.2, store the block in their local DAG, and (when they themselves propose at the next round) reference it as a parent of their own block.

**(L5)** As more rounds elapse, the Mysticeti commit rule of Section 10.3 eventually classifies the block's containing leader slot as either *to-commit* or *to-skip*. If *to-commit*, the leader's causal history is linearized into a deterministic transaction order and emitted to the execution engine.

**(L6)** The execution engine applies each transaction in order, recomputing the MuHash state commitment incrementally and producing an effect record per transaction (Section 6). It compares the resulting state_root against the value carried in the committed block; on mismatch, the validator halts and raises a fork alert.

**(L7)** The transaction is now *final*: its effects are durably persisted, the consumed UTXOs are removed from the storage layer, the new UTXOs are available for spending in

subsequent transactions, and the transaction's effect digest is reachable from the block's effects_root for light-client verification.

Steps L1–L4 require time bounded by the network round-trip; steps L5–L7 require the consensus commit delay, which under good network conditions is three additional rounds.

## 8. Validator Persistent State

Every MISAKA validator maintains a small set of persistent stores. We define each store, its key and value types, and its consistency requirements. The naming convention follows Sui [9, §4.2]: a store named $X_v$ is the local copy of $X$ held by validator $v$.

### 8.1. The UTXO store

Store name: $UTXO_v$. Key type: OutPoint = (Hash, U32). Value type: TxOut = (U64, LockScript). Semantics: maps every currently unspent output to its value and lock script.

$UTXO_v$ is the only store whose contents are necessary *and* sufficient to determine whether a given transaction is valid: a transaction is valid against $UTXO_v$ if and only if every consumed outpoint is present in the store and every unlock witness verifies. The store is updated atomically in lockstep with the MuHash accumulator: each insertion is paired with an accumulator multiplication, each deletion with an accumulator division, both committed inside the same storage write transaction. There is no point in time at which the store and the accumulator can disagree.

### 8.2. The DAG store

Store name: $DAG_v$. Key type: BlockID = Hash. Value type: Block (defined in Section 10.1). Semantics: holds every block validator $v$ has received and accepted as well-formed, indexed by content hash.

The DAG store is append-only within a single epoch: blocks are added but never removed or mutated. At epoch boundaries the store is pruned by the reconfiguration protocol (Section 11) to retain only the data needed for subsequent state-sync requests, governed by the retention_rounds parameter (default 10000).

### 8.3. The committed-leader log

Store name: $L_v$. Key type: U64 (commit index). Value type: BlockID. Semantics: a totally ordered sequence of leader block identifiers that $v$'s consensus engine has decided to commit, indexed by their position in the global commit order.

The agreement property of Byzantine Atomic Broadcast (Section 2.6) is the statement that, for any two honest validators $v_i$ and $v_j$, the contents of $L_{v_i}$ are a prefix of $L_{v_j}$ or vice versa. This is the single property the consensus engine of Section 10 must guarantee.

### 8.4. The validator-set store

Store name: $VS_v$. Key type: Epoch. Value type: Committee = (Vec<ValidatorID>, Vec<U128>, Vec<MLDSAPubKey>). Semantics: for each epoch, the ordered list of active validator identities, their active stakes in base units, and their operator public keys.

The validator-set store is updated exactly once per epoch by the reconfiguration protocol (Section 11). It is the source of truth for stake-weighted quorum calculations: every quorum check made by the consensus engine reads from $VS_v[e]$ for the current epoch $e$.

### 8.5. The supply tracker

Store name: $Supply_v$. Singleton (no key). Value type: SupplyTracker = (total: U128, circulating: U128, staked: U128, burned: U128, max: U128). Semantics: the current monetary state of the chain.

Every coinbase transaction increments total and circulating by its newly emitted amount; every stake operation moves value between circulating and staked; every fee burn (if any) increments burned. All updates use checked U128 arithmetic and are committed inside the same storage transaction as the corresponding UTXO write. The constraint total ≤ max is enforced on every mint; an attempted mint that would violate it returns SupplyError::ExceedsCap and the containing block is rejected.

**Snapshot durability.** $Supply_v$ is persisted on every commit, not only at checkpoint boundaries. A validator that cannot load this store on startup refuses to start rather than defaulting to a zero-initialized tracker, which would permit a restarted validator to re-emit already-committed rewards. This is contribution C3 of Section 1.1, and Theorem 7 of Section 15 proves it formally.

### 8.6. The write-ahead log

Store name: $WAL_v$. Append-only sequence of records. Each record carries a monotonic sequence number, a record type tag, the record payload, and a BLAKE3 checksum. The WAL is the only store written on the consensus critical path; the other stores are updated lazily from WAL records during a recovery sweep on startup. This follows the Mysticeti reference design [2, §VI] and allows the consensus engine to commit a block in a single vectored write rather than a multi-store transaction.

## 9. Authority Base Operation

An *authority* in MISAKA is a single honest validator considered in isolation: its job is to apply a small set of well-defined operations to its local state in response to incoming messages, in such a way that the global interaction of all honest authorities provides Byzantine Atomic Broadcast. This section specifies those operations as numbered procedures.

We use the following notation throughout. $v$ is the local authority. $e$ is the current epoch. $UTXO_v$, $DAG_v$, $L_v$, $VS_v$, $Supply_v$ are the persistent stores of Section 8. $S_e(\cdot)$ and $T_e$ are the per-validator and total active stake of Section 2.1.

### 9.1. Admitting a transaction

When the local RPC interface or the gossip layer hands a transaction tx to the authority, the following procedure decides whether to admit it to the mempool. The procedure is total: every input either succeeds or returns a deterministic error code.

**Procedure 1: process_transaction(tx)**

```
1. if not well_formed(tx): return Err(MALFORMED)
2. if tx.chain_id ≠ self.chain_id: return
Err(CHAIN_ID)
3. for each input i in tx.inputs:
    a. let utxo := UTXO_v[i.outpoint]
    b. if utxo = ⊥: return Err(MISSING_INPUT)
    c. let z := sighash(tx, i, utxo.lock)
    d. if not script_eval(utxo.lock, i.unlock,
z):
        return Err(BAD_WITNESS)
4. let in_sum := Σ UTXO_v[i.outpoint].value for i
in tx.inputs
5. let out_sum := Σ o.value for o in tx.outputs
6. if in_sum < out_sum + tx.fee: return
Err(INSUFFICIENT)
7. if mempool.contains(txid(tx)): return
Ok(DUPLICATE)
8. mempool.insert(tx, fee_rate := tx.fee /
size(tx))
9. gossip.broadcast(tx)
10. return Ok(ADMITTED)
```

process_transaction is read-only with respect to $UTXO_v$: it does not consume the inputs, because the transaction has not yet been ordered by consensus and may still conflict with another transaction in a different proposer's block. The actual consumption happens in execute_block below.

### 9.2. Receiving a block

When the gossip layer delivers a signed block b, the authority validates it and inserts it into the local DAG. The procedure rejects any block that violates the structural rules of Section 10.2 but does not yet decide whether the block will be committed; that decision is made by try_commit below.

**Procedure 2: process_block(b)**

```
1. if not well_formed_block(b): return
Err(MALFORMED)
2. if b.epoch ≠ self.epoch: return
Err(WRONG_EPOCH)
3. let pk := VS_v[b.epoch].pk_of(b.author)
4. if pk = ⊥: return Err(UNKNOWN_AUTHOR)
5. if not Verify_pk(block_sighash(b),
b.signature):
    return Err(BAD_SIGNATURE)
6. for each parent p in b.parents:
    a. if DAG_v[p] = ⊥: request p; suspend(b)
7. if b.parents.first ≠
latest_self_parent(b.author):
    return Err(BAD_SELF_PARENT)
8. if not stake_quorum(b.parents.authors at round
b.round-1):
    return Err(INSUFFICIENT_PARENTS)
9. DAG_v.insert(block_id(b), b)
10. WAL_v.append(BlockReceived { id :=
block_id(b) })
11. try_commit()
12. return Ok(ACCEPTED)
```

Step 6 (*request and suspend*) is the standard DAG synchronization mechanism: a block whose parents are not yet locally available is parked in a side queue and re-evaluated when the missing parents arrive. Step 7 enforces the first-self-parent rule of Section 10.2 B3, which is what makes the implicit certification structure consistent across forks. Step 8 is the stake-weighted analogue of Mysticeti's “at least $2f + 1$ parents” rule and uses the quorum predicate of Section 2.5.

### 9.3. Proposing a block

Once the authority has accepted a stake quorum of blocks at round $r - 1$, it constructs and broadcasts its own block at round $r$:

```
Procedure 3: propose_block(r)
1. let parents := select_parents(r) # see §10.2
2. let payload :=
mempool.drain_top(MAX_BLOCK_BYTES)
3. let b := Block {
    author := self.id,
    round := r,
    epoch := self.epoch,
    parents := parents,
    payload := payload,
    timestamp := now(),
    state_root := UTXO_v.muhash(),
    effects_root:= last_committed_effects_root,
    signature := ⊥,
  }
4. b.signature := Sign_sk(block_sighash(b))
5. DAG_v.insert(block_id(b), b)
6. WAL_v.append(BlockProduced { id := block_id(b)
})
7. gossip.broadcast(b)
```

The signing operation in step 4 is delegated to a dedicated worker thread in the production implementation to keep the consensus thread non-blocking; the abstract procedure shown here is sequential for clarity. The state_root field carries the MuHash of the UTXO set *as of the last committed leader*, not as of the block being proposed; the latter would force the proposer to predict the order in which concurrent proposals will be linearized, which is impossible without committing first.

### 9.4. Committing a leader and executing

After every successful process_block, the authority invokes try_commit, which applies the Mysticeti decision rule of Section 10.3 to attempt to advance the committed-leader log $L_v$:

```
Procedure 4: try_commit()
1. let new_commits := mysticeti_decide(DAG_v,
L_v)
2. for each leader_id in new_commits:
    a. let leader := DAG_v[leader_id]
    b. let txs :=
linearize(causal_history(leader))
    c. execute_block(leader, txs)
    d. L_v.append(leader_id)
    e. WAL_v.append(LeaderCommitted {
        id := leader_id,
        state_root := UTXO_v.muhash(),
    })
```

linearize is the deterministic post-order traversal of the leader's causal history specified in Section 10.4. execute_block applies each transaction in turn:

```
Procedure 5: execute_block(leader, txs)
1. let fee_pool := 0
2. for each tx in txs:
    a. if revalidate(tx) = Err(_):
        effects.append(Failure(tx))
        continue
    b. for each i in tx.inputs:
        UTXO_v.remove(i.outpoint)
    c. for each (j, o) in enumerate(tx.outputs):
        UTXO_v.insert((txid(tx), j), o)
    d. fee_pool += tx.fee
    e. effects.append(Success(tx))
3. let cb := build_coinbase(leader, fee_pool,
Supply_v)
4. for each (j, o) in enumerate(cb.outputs):
    UTXO_v.insert((txid(cb), j), o)
5. Supply_v.mint(cb.minted)? # may return
ExceedsCap
6. let local_root := UTXO_v.muhash()
7. if local_root ≠ leader.state_root:
    halt(); raise ForkAlert(local_root,
leader.state_root)
8. WAL_v.append(BlockExecuted {
    leader := leader.id,
    effects_root := merkle(effects),
  })
```

Step 2a (*revalidate*) is necessary because Procedure 1 admitted the transaction against an earlier UTXO state; by the time the transaction is executed, an earlier transaction in the same linearized order may have already consumed one of its inputs, in which case the transaction is committed with status Failure per Section 6.4. Step 7 is the independent state-root verification that prevents a Byzantine proposer from committing an arbitrary post-state.

## 10. Narwhal/Bullshark Consensus

MISAKA's consensus engine is a port of the Mysticeti uncertified DAG protocol [2] to a UTXO ledger and ML-DSA-65 signatures. This section gives the operational specification of the protocol; the security analysis is deferred to Section 15. We restate enough of the Mysticeti commit rule that this document is self-contained, but for the full proofs we refer to [2, §V].

### 10.1. Block structure

| Field | Type | Description |
|---|---|---|
| author | ValidatorID | Index of the proposing validator in V_e. |
| round | Round | Logical round number, monotonically increasing. |

| | | |
|---|---|---|
| epoch | Epoch | Epoch in which the block is valid. |
| parents | Vec<BlockID> | References to parent blocks; the first is the author's own previous block. |
| payload | Vec<Tx> | Ordered list of transactions to be included. |
| timestamp | U64 | Wall-clock proposal time, monotonic per author. |
| state_root | Hash | MuHash of the UTXO set as of the last committed leader. |
| effects_root | Hash | Merkle root of effect digests of the last committed leader. |
| signature | MLDSASig | ML-DSA-65 signature by the author over all preceding fields. |

*Table 7: Block structure.*

## 10.2. Block validity rules

A block b is *valid* at the local DAG of validator $v$ if and only if all of the following hold:

**(B1)** The signature verifies under the author's operator public key as recorded in $VS_v$[b.epoch].

**(B2)** Every parent identifier in b.parents is present in $DAG_v$, and every referenced parent has a round number strictly less than b.round.

**(B3)** The first parent in b.parents is the most recent block authored by b.author known to $v$. (First-self-parent rule.)

**(B4)** The set of authors of round-(b.round − 1) parents in b.parents constitutes a stake quorum (Section 2.5) under $S_{b.epoch}$.

**(B5)** The total serialized size of b.payload does not exceed MAX_BLOCK_BYTES.

Rules B1–B4 are exactly the Mysticeti validity rules with stake-weighted quorums substituted for the count-weighted “$2f + 1$” in B4. Rule B5 is a local resource bound that is enforced by all honest validators with the same parameter, so the rule is consensus-visible even though it is not part of the BFT abstraction.

## 10.3. Commit rule

MISAKA reuses the Mysticeti three-step commit rule [2, §III-B]. We sketch it here in stake-weighted form. A *proposer slot* is a pair ($v$, $r$) consisting of a validator and a round; under MISAKA's default configuration there are two proposer slots per round, ordered deterministically by a round-robin permutation seeded by the epoch number. Each slot is initially in state undecided.

Given a block b at round $r$ proposed by validator $v$, we say:

**(D1)** A block b' at round $r + 1$ *supports* b if a depth-first search from b' following parent references encounters b as the first block authored by $v$ at round $r$.

**(D2)** Block b is *certified* if a stake quorum of round-($r + 1$) blocks support it.

**(D3)** A block c at round $r + 2$ is a *certificate* for b if its causal history contains a stake quorum of supporting blocks.

The commit rule is then:

**Direct decision rule.** A slot is marked to-commit if a stake quorum of round-($r + 2$) blocks are certificates for some proposal in the slot. It is marked to-skip if for every proposal in the slot, a stake quorum of round-($r + 1$) blocks fail to support it. Otherwise the slot remains undecided.

**Indirect decision rule.** If the direct rule fails to decide a slot $s$, the validator searches forward in the proposer-slot ordering for the first slot $s'$ at round $r' > r + 2$ that is already to-commit or undecided. If $s'$ is undecided, then $s$ remains undecided. If $s'$ is to-commit with anchor block a, then $s$ is marked to-commit if and only if a's causal history contains a certificate for some proposal in $s$; otherwise $s$ is marked to-skip.

**Commit sequence.** After processing all slots, the validator iterates the proposer-slot ordering, appending each to-commit block to $L_v$ and skipping each to-skip slot, until it encounters the first still-undecided slot, at which point it stops.

Under good network conditions the direct rule decides every slot in three message delays (propose → support → certify), matching the Byzantine lower bound of Martin and Alvisi [10]. Under crash failures or asynchrony the indirect rule defers the decision by at most a few additional rounds; Section 15 gives the formal liveness argument.

### 10.4. Linearization

When a leader block b is committed, its causal history must be linearized into a deterministic transaction order. MISAKA uses the same linearization as Mysticeti: a post-order traversal of the sub-DAG rooted at b consisting of blocks not yet committed by any prior leader, with ties between siblings broken by validator index in $VS_v$. Within each block, transactions are taken in the order they appear in payload. Duplicate transactions encountered later in the traversal are dropped without error.

This linearization is purely a function of the committed DAG and the epoch's validator-set ordering, both of which are agreed by all honest validators (Sections 8.2 and 8.4). It therefore produces bit-identical transaction sequences across all honest validators, which is the precondition for bit-identical state_root values in Procedure 5.

### 10.5. Liveness timeout

Like Mysticeti, MISAKA relies on a single liveness timeout for the primary proposer of each round. After accepting a stake quorum of blocks at round $r - 1$, an honest validator waits up to LEADER_TIMEOUT = 3Δ for the primary proposer's block at round $r$ to arrive before proposing its own round-$r$ block. The default configured value is one second on the testnet. This guarantees that, after GST, an honest primary proposer's block will be referenced as a parent by all other honest validators, which in turn guarantees that it will be certified and eventually committed by the direct rule.

## 11. Epoch Change and Reconfiguration

An *epoch* is a maximal contiguous range of rounds during which the committee $C_e$ remains fixed. At an epoch boundary the committee may change: validators may join, leave, or have their stake updated. This section specifies how epoch transitions are detected, executed, and persisted in such a way that the chain remains live and safe across the boundary.

### 11.1. Epoch length

An epoch ends after a configurable wall-clock duration EPOCH_DURATION_SECS, which on the production configuration is set to 30 days. The exact boundary is the first leader committed at or after the epoch deadline; this rule depends only on committed timestamps (Section 10.1) and not on validator wall clocks, so all honest validators agree on the boundary deterministically.

### 11.2. Validator selection

At every epoch boundary, the active set $V_e$ for the next epoch is recomputed from the on-chain validator registry. The selection rule is:

**(R1)** Filter the registry to candidates with active_stake ≥ MIN_STAKE_ACTIVE (= $10^7$ MISAKA) and score ≥ MIN_ACTIVE_SCORE (= 0.95). Candidates that fail either threshold are excluded.

**(R2)** Compute each candidate's *reward weight* as

$$w_i = \mathrm{isqrt}(\mathrm{stake}_i) \cdot \mathrm{score}_i$$

where isqrt is the integer square root of Section 3.4 (deterministic, branchless, U128).

**(R3)** Sort candidates in descending order of $w_i$, breaking ties by ascending validator ID. Take the top 21 to form $V_{e+1}$.

The square-root weighting is contribution C4 of Section 1.1: a candidate with 100× the stake of another receives only 10× the reward weight, substantially reducing the marginal return on stake concentration while preserving strict monotonicity in stake.

The score is a smoothed exponentially-weighted moving average of per-epoch performance observations:

$$\mathrm{score}_t = \alpha \cdot \mathrm{score}_{t-1} + (1 - \alpha) \cdot \mathrm{obs}_t$$

with α = 0.9 in the reference implementation. The per-epoch observation $obs_t$ is the fraction of the epoch's leader rounds in which the validator's block was successfully committed, computed in fixed-point arithmetic with 4 decimal digits of precision. MISAKA does *not* implement slashing: a validator that drops below MIN_ACTIVE_SCORE is rotated out of the active committee at the next epoch boundary but retains its stake and may re-enter the active set when its score recovers.

### 11.3. Voting weight versus reward weight

It is important to distinguish two uses of the weight function. *Voting weight* in the consensus engine is the validator's active_stake directly, in base units; this is what enters the stake-quorum predicate of Section 2.5 and what determines whether a set of validators forms a quorum. *Reward weight*, by contrast, is isqrt(active_stake) × score and is used only for (a) selecting the top 21 active validators and (b) dividing the per-epoch reward pool among them. The square-root function therefore affects *income* and *committee membership*, but not *safety*: safety is governed entirely by the linear stake quorum, which is the BFT-correct quantity.

### 11.4. Epoch transition procedure

The epoch transition is itself a deterministic procedure executed by every honest validator at the same point in the committed leader sequence:

**Procedure 6: epoch_transition(e)**

```
1. let registry :=
validator_registry_state(UTXO_v)
2. let new_committee :=
select_committee(registry) # §11.2
3. let next_epoch := e + 1
4. VS_v.insert(next_epoch, new_committee)
5. let final_root := UTXO_v.muhash()
6. WAL_v.append(EpochClosed {
    epoch := e,
    final_state_root := final_root,
    next_committee := hash(new_committee),
  })
7. self.epoch := next_epoch
8. dag_engine.reset_for_epoch(next_epoch,
new_committee)
9. retire_old_dag_blocks(retention_rounds)
```

The validator registry of step 1 is itself part of the UTXO state: stake-bond and stake-unbond operations are ordinary MISAKA transactions whose effects update designated registry outputs. This means the input to select_committee is fully determined by the committed transaction order, so all honest validators compute identical new_committee values without any explicit cross-validator coordination.

### 11.5. Cross-epoch safety

Because MISAKA is a pure consensus chain (it has no consensusless fast path), the safety property required at epoch boundaries is the simpler of the two cases discussed in [2, §V-B]: every transaction committed in epoch $e$ must remain committed in epoch $e + 1$. This is immediate from the construction of epoch_transition: the epoch closes by appending an EpochClosed record to the WAL after a particular leader has been committed and executed, and the next epoch starts from exactly that committed state. There is no race between fast-path finalization and consensus finalization to manage. The final_state_root field carried in the EpochClosed record allows a joining validator in the new epoch to verify that it starts from the correct state without replaying the entire epoch's history.

## 12. Light Clients

A *light client* is a participant that wishes to verify specific facts about the MISAKA ledger—the inclusion of a transaction, the value of a UTXO, the contents of an address—without storing or executing the entire chain. Light clients are essential for mobile wallets, browser extensions, exchange withdrawal verifiers, and any application that cannot afford the bandwidth and storage costs of a full validator. This section specifies the intended light-client protocol for MISAKA. We note that the v1 reference implementation does not yet ship a light client; the design described here is the target for v1.1.

### 12.1. Header chain

A light client tracks the *header chain*: the sequence of committed leader-block headers, where a header is the set of fields from Table 7 of Section 10.1 *excluding* the payload. A header is approximately 4 KB on the wire (3.3 KB for the ML-DSA-65 signature plus parent hashes, roots, and metadata), compared to a full block which may be hundreds of kilobytes. A light client downloads only headers, plus selective Merkle proofs into the bodies of headers it cares about.

### 12.2. Trust anchor: the validator set

Light clients begin from a trusted snapshot of the genesis $VS_0$: the initial set of validator identities and ML-DSA-65 public keys. This snapshot is distributed out-of-band (via the official release tarball, via signed update channels, or via comparison among multiple independent sources). Given this anchor, the light client follows the chain of EpochClosed records of Section 11.4. Each such record is signed (indirectly, by being committed in a leader block whose author signature the client can verify) and contains the hash of the next epoch's committee. The client therefore extends its trust forward one epoch at a time, never needing to re-verify earlier epochs.

This is structurally identical to the “trusted checkpoint” mechanism of Cosmos light clients and the “sync committee” mechanism of the Ethereum Beacon Chain. The post-quantum specialization is purely in the signature scheme: the same recursive trust structure works regardless of whether the underlying signatures are Ed25519, BLS, or ML-DSA-65.

### 12.3. Quorum certificate verification

A light client verifies that a leader block has been committed by checking that it carries individual ML-DSA-65 signatures from a stake quorum of the current epoch's committee. Concretely, an inclusion proof for a transaction *tx* consists of:

**(P1)** The committed leader header h containing effects_root.

**(P2)** A Merkle inclusion proof for the effect digest of *tx* against h.effects_root.

**(P3)** A set of subsequent block headers carrying ML-DSA-65 signatures whose authors collectively form a stake quorum on h under *VS*[h.epoch]. (One round of certification and one round of supports, per the Mysticeti commit rule of Section 10.3, suffice.)

The light client verifies (P3) by checking each ML-DSA-65 signature against the corresponding operator public key in *VS*, summing the corresponding stake values from the trusted committee, and checking that the sum exceeds $(2 / 3) \cdot T_e$. Verification cost is dominated by the signature checks: at 21 validators and roughly 14 signatures needed to reach a stake quorum, the worst case is 14 ML-DSA-65 verifications, approximately 2 ms on a modern mobile CPU.

### 12.4. Absence of signature aggregation

BLS-based chains use signature aggregation to compress a quorum of validator signatures into a single 96-byte aggregate, reducing both header size and verification cost. No equivalent aggregation scheme exists for ML-DSA-65 at the time of this draft: the structural proofs that make BLS aggregation work do not transfer to lattice schemes, and the candidate post-quantum aggregation schemes in the research literature are either not yet standardized or have prohibitively large public parameters. MISAKA therefore carries one independent ML-DSA-65 signature per validator per quorum certificate. We accept this overhead as the cost of post-quantum security and discuss its quantitative impact in Section 14.

## 13. Tokenomics

MISAKA's monetary policy is governed by four parameters: a hard maximum supply, a decaying inflation schedule with a permanent floor, a fee-rate-priority transaction fee model, and a snapshot-durable on-chain supply tracker. This section specifies each in turn.

### 13.1. Maximum supply and units

The hard maximum supply is MAX_SUPPLY = $10^{10}$ MISAKA. One MISAKA equals $10^9$ base units, so MAX_SUPPLY is $10^{19}$ base units. The constraint

$$\texttt{Supply}_v\texttt{.total} \le \texttt{MAX_SUPPLY}$$

is enforced on every mint operation, as specified in Section 8.5. A mint that would violate this constraint returns an error and the containing block is rejected by all honest validators.

### 13.2. Emission curve

MISAKA uses a piecewise linear, integer-arithmetic inflation curve. Let $r(y)$ denote the annual inflation rate in basis points for chain year $y$. Then

$$r(y) = \max(R_0 - D \cdot y,\ R_{floor})$$

with $R_0$ = 300 bps (3.00%), $D$ = 50 bps (0.50% per year), and $R_{floor}$ = 100 bps (1.00%). The floor is reached at

$$y_{floor} = (R_0 - R_{floor}) / D = (300 - 100) / 50 = 4,$$

i.e., year 4 of the chain. From year 4 onward, inflation remains at 1.00% indefinitely.

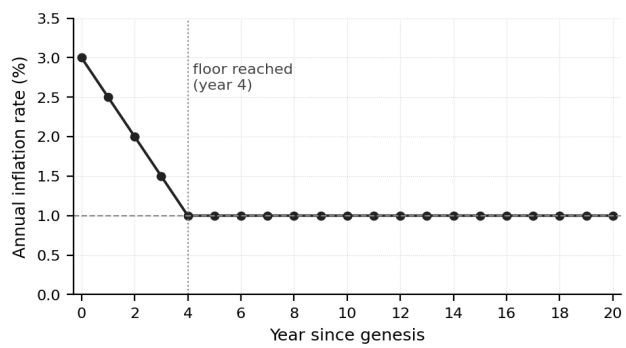


*Figure 1: MISAKA annual inflation curve. The rate decays from 3.00% to 1.00% over four years and remains at the floor indefinitely.*

Per-epoch emission is computed in integer arithmetic with no floating point:

$$\texttt{emission(y)} = \lfloor (S_y \cdot r(y)) / (10^4 \cdot E) \rfloor$$

where $S_y$ is the total supply at the start of year $y$, $r(y)$ is the annual rate in basis points, and $E$ is the number of epochs per year. The division order—basis points first, then epochs—is chosen to minimize precision loss in the u128 intermediates. The implementation uses checked_mul for the supply×rate product and falls back to a divide-first path on overflow, ensuring deterministic behavior even at extreme supply values.

### 13.3. Coinbase distribution

Each committed leader block carries exactly one coinbase transaction, constructed deterministically from the committed sub-DAG and the supply tracker. The coinbase produces two outputs:

**Proposer output:** the per-epoch inflation emission for the leader's slot, plus the proposer's share of fees from the linearized causal history of the leader.

**Treasury output:** a configurable fraction of the fees, sent to a designated treasury address governed by on-chain parameters. The treasury fraction defaults to zero on the testnet and is reserved for future ecosystem funding.

Because both outputs are derived deterministically from data available to all validators (the leader header, the linearization order, and the on-chain parameter set), two honest validators independently produce bit-identical coinbase transactions.

### 13.4. Fee model

MISAKA uses a fee-rate-priority mempool model, in the spirit of Bitcoin and Kaspa. Each transaction carries an explicit fee field equal to the difference between input and output values; the validator's mempool computes the fee rate as fee / size and orders pending transactions by descending rate. There is no base-fee mechanism in the EIP-1559 sense, no fee burning, and no per-block gas target. The motivation for this minimalism is twofold: first, EIP-1559's base-fee computation introduces a new piece of consensus-visible state that must be agreed upon every block, which interacts non-trivially with DAG-based commit rules; second, in a chain whose throughput vastly exceeds current demand (Section 14), the dynamic-base-fee machinery would simply pin the base fee at its minimum value, providing no benefit.

Fees are paid to the proposer and (optionally) the treasury via the coinbase output of Section 13.3. MISAKA does not burn fees in the steady state. The long-run economic incentive of validators is therefore the sum of the 1% perpetual inflation floor and the transaction fee revenue, with the latter expected to dominate as adoption grows.

### 13.5. Staking and bonding

Stake operations are ordinary MISAKA transactions whose outputs commit to a designated *stake script*: a lock that requires both an ML-DSA-65 signature from the stakeholder and the elapsed time of an unbonding period. Bonding moves value from $Supply_v$.circulating to $Supply_v$.staked; unbonding initiates a UNBOND_PERIOD_EPOCHS waiting period after which the value moves back. During the waiting period the stake is not counted toward the validator's voting weight or reward weight, but is also not slashable (MISAKA does not slash; see Section 11.2).

## 14. Scaling and Latency

This section gives an analytical treatment of the throughput and latency upper bounds of MISAKA. We deliberately defer empirical wide-area measurements to a companion paper based on testnet deployments; the figures below are derived from message and signature sizes alone and represent the constant factors imposed by the post-quantum design over the underlying Mysticeti baseline.

### 14.1. Block-level overhead

Recall from Section 3.1 that an ML-DSA-65 signature is 3309 bytes and a public key is 1952 bytes. An MISAKA block (Table 7) carries exactly one signature and references its parents by 32-byte hashes. For a 21-validator committee with a stake quorum of approximately 14 parents, the fixed cryptographic overhead per block is

```
3309 (sig) + 14×32 (parents) + 64 (roots) ≈ 3.8 KB.
```

This is the wire-size floor of a block, regardless of transaction payload. By comparison, an Ed25519-signed Mysticeti block has a wire-size floor of approximately 0.6 KB. The MISAKA overhead is therefore approximately 3.2 KB per block, dominated by the single ML-DSA-65 signature.

### 14.2. Network-level overhead

Each validator broadcasts one block per round to all $n - 1$ other validators. At $n = 21$ and a 2-second round, each validator therefore sends 20 outbound copies of its own block per round and receives 20 inbound copies of others' blocks. The signature traffic alone is

```
20 · 3309 B / 2 s ≈ 33 KB/s ≈ 264 kbps.
```

Doubling for the inbound direction yields approximately 528 kbps of pure signature traffic per validator, which is negligible on the 1 Gbps links typically available to validator hardware. The corresponding Ed25519 figure is roughly

10 kbps; MISAKA's signature-traffic overhead is approximately 50× higher in absolute terms but remains well below any realistic bandwidth bottleneck.

### 14.3. Verification CPU

ML-DSA-65 signature verification is approximately 1.4× the cost of Ed25519 verification on commodity x86 hardware according to the FIPS 204 reference implementation [3]. At 21 validators and a 2-second round, each validator must verify 20 inbound block signatures every two seconds, plus the per-input transaction signatures. For block-level verification this is approximately 10 verifications per second, well within a single CPU core. Per-transaction signature verification scales linearly with throughput and parallelizes across cores.

### 14.4. Latency

The Mysticeti commit rule of Section 10.3 commits a leader block in three message delays under good network conditions, matching the Byzantine lower bound [10]. MISAKA inherits this latency. The substitution of ML-DSA-65 for Ed25519 adds a small constant per signing and verification operation but does not change the round structure. We therefore expect MISAKA's wide-area commit latency to track the Mysticeti baseline reported in [2, Table I] (650 ms P50 / 975 ms P95 at 137 validators) within the constant factors of Sections 14.2 and 14.3. Empirical confirmation is the subject of forthcoming testnet measurements.

### 14.5. Throughput

Throughput in a Mysticeti-style DAG is bounded by the product of (i) the number of validators that may propose in a round, (ii) the maximum payload size per block, and (iii) the round rate. With 21 validators each proposing once per 2-second round and a maximum block payload of approximately 1 MB, the raw on-wire throughput is on the order of 10 MB/s, corresponding to several tens of thousands of transactions per second at typical UTXO transaction sizes. This is several orders of magnitude above current blockchain demand and is not a binding constraint for the foreseeable future.

## 15. Security Analysis

This section argues the safety, liveness, and post-quantum properties of MISAKA. Safety and liveness of the consensus engine are inherited from Mysticeti [2] with the substitutions of stake-weighted quorums (Section 2.5) and ML-DSA-65 signatures (Section 3.1). We restate the four BAB properties as theorems and sketch their proofs by reference to the corresponding Mysticeti lemmas.

### 15.1. Safety

**Theorem 1 (Agreement).** If two honest validators $v_i$ and $v_j$ deliver transactions $tx$ and $tx'$, then $v_i$'s delivered sequence is a prefix of $v_j$'s delivered sequence or vice versa.

*Proof sketch.* By Lemmas 2–7 of [2], applied with the stake-weighted quorum predicate of Section 2.5 in place of the count-weighted "$2f + 1$" predicate. The Mysticeti proofs use quorum intersection only, which is preserved by the stake-weighted formulation: any two stake quorums intersect in a set of validators whose total stake exceeds $T_e / 3$ and therefore contains at least one honest validator under the bound of Section 2.2. The substitution of ML-DSA-65 for Ed25519 affects only the verify_signature primitive used at step 5 of Procedure 2 and does not change the structure of the proof; we assume ML-DSA-65 is EUF-CMA secure under MLWE/MSIS. The remaining steps—committed leader linearization (Section 10.4) and execution (Procedure 5)—are deterministic functions of the committed leader log $L_v$, which is consistent across honest validators by Lemma 7 of [2]. Therefore the delivered sequences are consistent. □

**Theorem 2 (Integrity).** Each honest validator delivers each transaction at most once, and only if some validator previously broadcast it.

*Proof sketch.* The linearization of Section 10.4 deduplicates transactions encountered more than once in the post-order traversal of a leader's causal history. A transaction can only enter $UTXO_v$ via Procedure 5 step 2c, which requires the transaction to have been included in the payload of some block b in the committed sub-DAG; that block was admitted to $DAG_v$ by Procedure 2, which verifies the author's signature on b; therefore some validator broadcast the transaction. □

**Theorem 3 (Validity).** If an honest validator broadcasts a well-formed transaction $tx$, then every honest validator eventually delivers $tx$, provided the network is in a synchronous period and $tx$ remains valid against the running state.

*Proof sketch.* By the gossip property of the P2P transport, $tx$ reaches every honest validator's mempool within bounded time after broadcast. By the liveness argument of Theorem 5 of [2] applied to the stake-weighted quorum, an honest proposer eventually includes $tx$ in a block that is committed by the Mysticeti commit rule. By Theorem 1, this commit is observed by every honest validator, which then delivers $tx$ via Procedure 5. □

**Theorem 4 (Total order).** If two honest validators deliver transactions $tx$ and $tx'$, they deliver them in the same relative order.

*Proof sketch.* The committed leader log $L_v$ is consistent across honest validators by Theorem 1, and the linearization function of Section 10.4 is deterministic. Therefore the deliverable order is identical on every honest validator. □

### 15.2. Post-quantum properties

**Theorem 5 (Post-quantum unforgeability).** Under the EUF-CMA security of ML-DSA-65 against polynomial-time quantum adversaries, no quantum-bounded adversary can produce a valid signature on behalf of an honest validator without access to that validator's secret key.

*Proof sketch.* Direct from the EUF-CMA assumption of Section 3.1. Any signature accepted by Procedure 2 step 5 must verify under the validator's operator public key recorded in $VS_v$; producing such a signature without the secret key contradicts EUF-CMA. □

**Theorem 6 (Post-quantum confidentiality of P2P transcripts).** Under the IND-CCA2 security of ML-KEM-768 against polynomial-time quantum adversaries and the AEAD security of ChaCha20-Poly1305 against classical adversaries, a network observer who records a complete P2P transcript and later acquires a CRQC cannot recover the plaintext of any honest-to-honest session.

*Proof sketch.* The session key derived by HKDF-SHA3-256 from the ML-KEM-768 shared secret is unrecoverable from the recorded ciphertext under IND-CCA2, even by a future quantum adversary, because IND-CCA2 against quantum adversaries is precisely the assumption of FIPS 203. Without the session key, the AEAD ciphertext under ChaCha20-Poly1305 is indistinguishable from random by any classical or quantum adversary in standard symmetric models. □

### 15.3. Snapshot durability of supply

**Theorem 7 (Cap enforcement under restart).** After any sequence of crash-restart events on validator $v$, the persisted $Supply_v$.total exactly equals the cumulative emission committed in $L_v$ up to the last fully executed leader. In particular, no restart can cause $v$ to re-emit already-committed rewards.

*Proof sketch.* Procedure 5 step 5 calls $Supply_v$.mint(·) *before* appending BlockExecuted to $WAL_v$ in step 8, and the mint is part of the same atomic storage transaction as the UTXO writes. On crash recovery, the WAL is replayed and the supply tracker is restored to exactly the state recorded by the last BlockExecuted entry. The startup protocol of Section 8.5 refuses to start if the tracker cannot be loaded, eliminating the failure mode in which a recovered validator reinitializes the tracker from a genesis value. □

## 16. Related Work

**DAG-based BFT consensus.** MISAKA's consensus engine descends from the line of work on structured DAG consensus: HashGraph [11], DagRider [12], Narwhal/Tusk [5], Bullshark [6], and Mysticeti [2]. The defining innovation of Mysticeti, which MISAKA inherits, is the use of *uncertified* DAG blocks: instead of certifying each block via a separate consensus instance, certification is implicit in the DAG structure and the commit rule interprets it on the fly. This reduces commit latency from approximately six message delays (Bullshark) to three (Mysticeti, MISAKA), matching the Byzantine lower bound of Martin and Alvisi [10].

**Post-quantum blockchains.** Post-quantum cryptography in production blockchains is most commonly framed as a *future migration path* rather than a genesis property. Several research proposals discuss hybrid schemes that retain an elliptic-curve component as a transitional measure; we have argued in Section 1 that such hybrids defeat the post-quantum forward-secrecy goal because they remain vulnerable to harvest-now-decrypt-later attacks. The closest architectural peer that takes the “post-quantum from genesis” position is the Quantum Resistant Ledger (QRL), which uses XMSS hash-based signatures; QRL's consensus is proof-of-work, however, and its data model is account-based, so the comparison with MISAKA is structural rather than direct. To our knowledge, MISAKA is the first complete specification of a Layer 1 blockchain whose consensus, transport, and user signing scheme are all exclusively lattice-based.

**UTXO chains.** The transparent UTXO model originates with Bitcoin [8] and has been adopted by Litecoin, Monero (with privacy extensions), Cardano (extended UTXO), and Kaspa. MISAKA's script engine is Kaspa-compatible in the subset of opcodes implemented, with the addition of ML-DSA-65 signature opcodes (Section 4.4). Earlier MISAKA prototypes included ring-signature and stealth-address privacy features based on pairing-based cryptography; these were removed after internal review determined that pairings are themselves quantum-vulnerable and therefore incompatible with the chain's stated threat model.

**DPoS and stake weighting.** The 21-slot active committee draws from the EOS and Tron tradition. The square-root weighting rule is a deliberate departure: neither EOS nor Tron uses a sublinear function of stake. Quadratic voting [13] motivates the square-root choice as a means of reducing the marginal influence of large holders, and Cardano's Ouroboros protocol family [11] motivates the no-slashing choice on the grounds that slashing creates outsized incentives for centralized staking-as-a-service providers.

## 17. Conclusion

We have specified MISAKA Network: a Layer 1 blockchain whose consensus engine, peer-to-peer transport, and user-level signing scheme are all exclusively lattice-based, with explicit absence of elliptic-curve cryptography from the consensus critical path. MISAKA combines a transparent UTXO ledger authenticated by ML-DSA-65, a SHA3-256 MuHash incremental commitment over the unspent set, an ML-KEM-768 encrypted P2P transport, a Narwhal/Bullshark uncertified DAG consensus engine ported from Mysticeti, and a 21-slot DPoS committee with square-root reward weighting and a snapshot-durable supply tracker. The design takes the post-quantum threat model as a genesis property rather than a future migration item, and the reference implementation is currently live on a public testnet.

**Open problems.** Several questions remain for future work. First, a complete formal reduction of the entire MISAKA protocol—not only its consensus core—to the MLWE and MSIS hardness assumptions would strengthen the security argument. Second, an empirical wide-area evaluation on a 21-validator geo-distributed testbed is required to validate the analytical bounds of Section 14. Third, a production-ready light client matching the design of Section 12 must be implemented and audited. Fourth, on-chain governance for validator-set parameters and emission curves must be specified in a way that preserves stake durability across cryptographic parameter changes.

## References


[1] P. W. Shor, “Algorithms for quantum computation: discrete logarithms and factoring,” in *Proc. 35th FOCS*, 1994.

[2] K. Babel, A. Chursin, G. Danezis, A. Kichidis, L. Kokoris-Kogias, A. Koshy, A. Sonnino, and M. Tian, “Mysticeti: Reaching the latency limits with uncertified DAGs,” in *NDSS Symposium*, 2025.

[3] National Institute of Standards and Technology, *Module-Lattice-Based Digital Signature Standard*, NIST FIPS 204, 2024.

[4] National Institute of Standards and Technology, *Module-Lattice-Based Key-Encapsulation Mechanism Standard*, NIST FIPS 203, 2024.

[5] G. Danezis, L. Kokoris-Kogias, A. Sonnino, and A. Spiegelman, “Narwhal and Tusk: a DAG-based mempool and efficient BFT consensus,” in *EuroSys*, 2022.

[6] A. Spiegelman, N. Giridharan, A. Sonnino, and L. Kokoris-Kogias, “Bullshark: DAG BFT protocols made practical,” in *ACM CCS*, 2022.

[7] National Institute of Standards and Technology, *SHA-3 Standard: Permutation-Based Hash and Extendable-Output Functions*, NIST FIPS 202, 2015.

[8] S. Nakamoto, “Bitcoin: A peer-to-peer electronic cash system,” 2008.

[9] S. Blackshear, A. Chursin, G. Danezis *et al.*, “Sui Lutris: A blockchain combining broadcast and consensus,” arXiv preprint, 2023.

[10] J. Martin and L. Alvisi, “Fast Byzantine consensus,” *IEEE TDSC*, vol. 3, no. 3, 2006.

[11] A. Kiayias, A. Russell, B. David, and R. Oliynykov, “Ouroboros: A provably secure proof-of-stake blockchain protocol,” in *CRYPTO*, 2017.

[12] I. Keidar, E. Kokoris-Kogias, O. Naor, and A. Spiegelman, “All you need is DAG,” in *PODC*, 2021.

[13] E. G. Weyl, “Quadratic voting and the public good,” *Public Choice*, vol. 172, 2017.

[14] L. Baird, “The Swirlds Hashgraph consensus algorithm: Fair, fast, Byzantine fault tolerance,” Swirlds Tech Report, 2016.

[15] M. Castro and B. Liskov, “Practical Byzantine fault tolerance,” in *OSDI*, 1999.

[16] National Institute of Standards and Technology, *SHA-3 Derived Functions: cSHAKE, KMAC, TupleHash, and ParallelHash*, NIST SP 800-185, 2016.


## Appendix A. Script Opcode Reference

Table A.1 lists the opcodes implemented by the MISAKA v1 script engine. The set is intentionally minimal: it supports the standard single-signature lock and a small number of stack-manipulation primitives, with no Turing-complete control flow.

| Opcode | Hex | Description |
|---|---|---|

| | | |
|---|---|---|
| `OP_0` | 0x00 | Push the empty byte string. |
| `OP_DUP` | 0x76 | Duplicate the top stack item. |
| `OP_HASH256` | 0xAA | Replace top item with its SHA3-256 hash. |
| `OP_EQUALVERIFY` | 0x88 | Pop two items; fail if not equal. |
| `OP_CHECKSIG_MLDSA` | 0xC0 | Pop (pk, sig); push 1 if Verify(pk, sighash, sig) else 0. |
| `OP_CHECKSIGVERIFY_MLDSA` | 0xC1 | As above, but fails the script on a 0 result. |

*Table A.1: MISAKA v1 script opcodes.*

## Appendix B. Constants

| Constant | Value |
|---|---|
| MAX_SUPPLY | $10^{10}$ MISAKA = $10^{19}$ base units |
| DECIMALS | 9 |
| MIN_STAKE_ACTIVE | $10^{7}$ MISAKA |
| MIN_STAKE_BACKUP | $10^{6}$ MISAKA |
| ACTIVE_SET_SIZE | 21 |
| MIN_ACTIVE_SCORE | 0.95 |
| EWMA_ALPHA | 0.9 |
| INITIAL_INFLATION_BPS | 300 (3.00%) |
| ANNUAL_DECAY_BPS | 50 (0.50% per year) |
| FLOOR_INFLATION_BPS | 100 (1.00%) |
| FLOOR_REACHED_AT_YEAR | 4 |
| EPOCH_DURATION_SECS | 30 × 24 × 3600 (30 days) |
| LEADER_TIMEOUT | 3Δ (≈1 second on testnet) |
| ROUND_TARGET_SECS | 2 |
| P2P_PORT (testnet) | 6690 |
| RPC_PORT (testnet) | 3001 |
| CHAIN_ID (testnet) | 2 |
| CHAIN_ID (mainnet) | 1 |

*Table B.1: MISAKA v1 protocol constants.*

## Appendix C. Acknowledgments

MISAKA Network builds directly on the Mysticeti reference implementation by the Mysten Labs team and on the line of DAG-based BFT consensus research by Danezis, Kokoris-Kogias, Sonnino, Spiegelman, and collaborators. The post-quantum primitives are due to the CRYSTALS-Kyber and CRYSTALS-Dilithium teams and to NIST. The MuHash construction is due to Maxwell, Poelstra, Seurin, and Wuille. Errors in adaptation are entirely the author's.